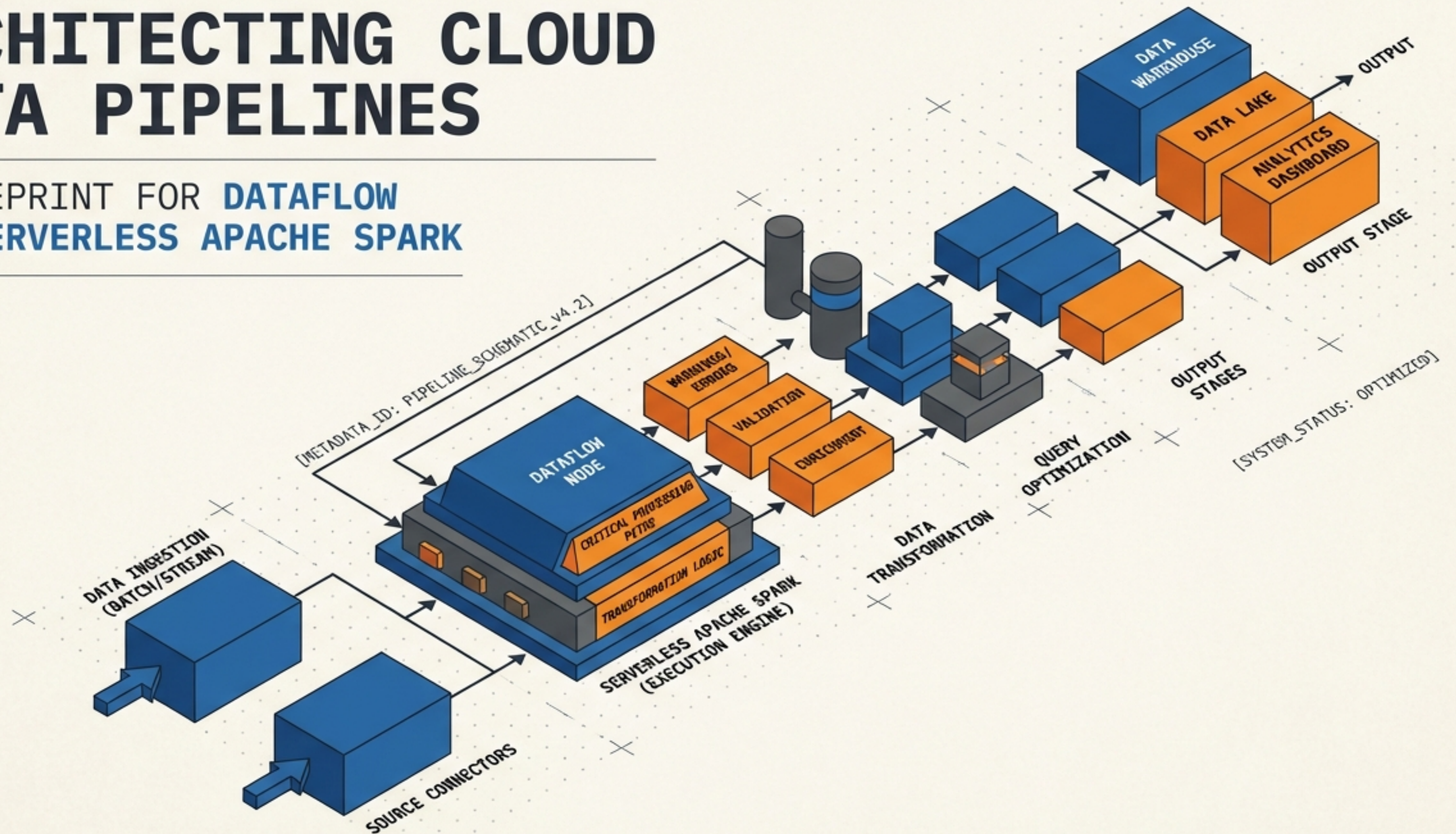


ARCHITECTING CLOUD DATA PIPELINES

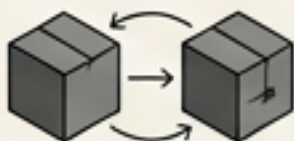
A BLUEPRINT FOR DATAFLOW AND SERVERLESS APACHE SPARK



THE OBJECTIVE: SCALABLE, SERVERLESS BATCH PROCESSING

THE CORE CONCEPT

Moving, transforming, and analyzing bounded datasets—data with a fixed size available in its entirety before processing begins.



THE CONSTRAINT

Executing complex operations (ETL, machine learning prep, querying) without the overhead of manually provisioning, tuning, or maintaining cluster infrastructure.



THE SOLUTION ENGINES

Google Cloud provides two primary vehicles for this task: Dataflow (powered by Apache Beam) and Serverless for Apache Spark.

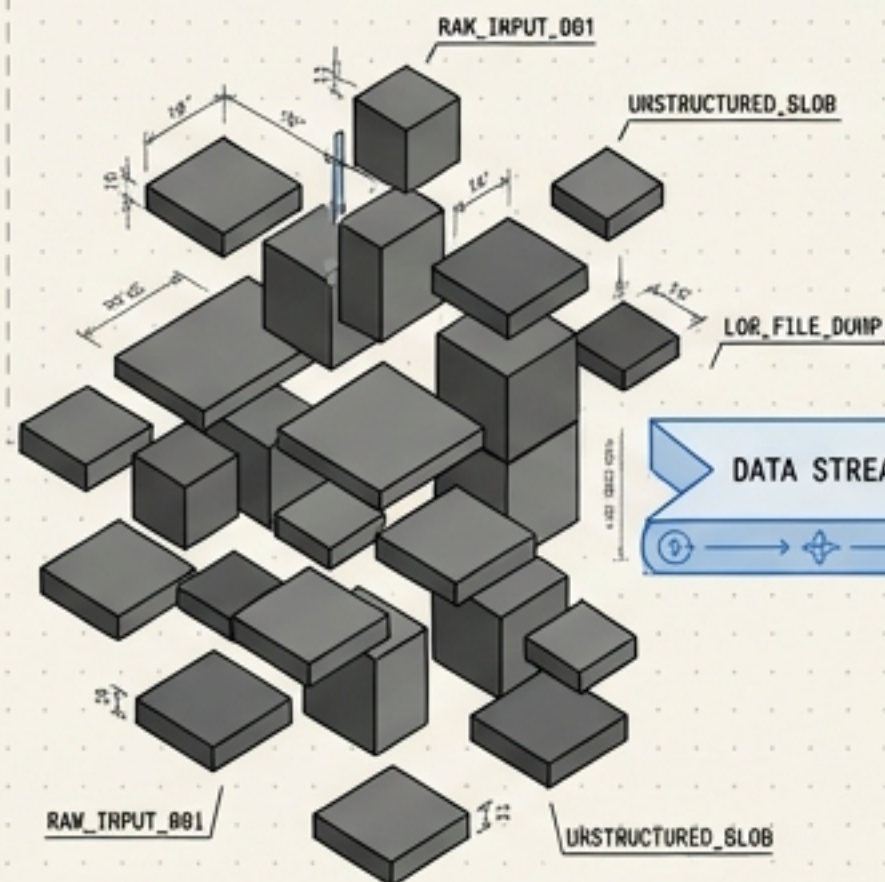


DATAFLOW
(Apache Beam)

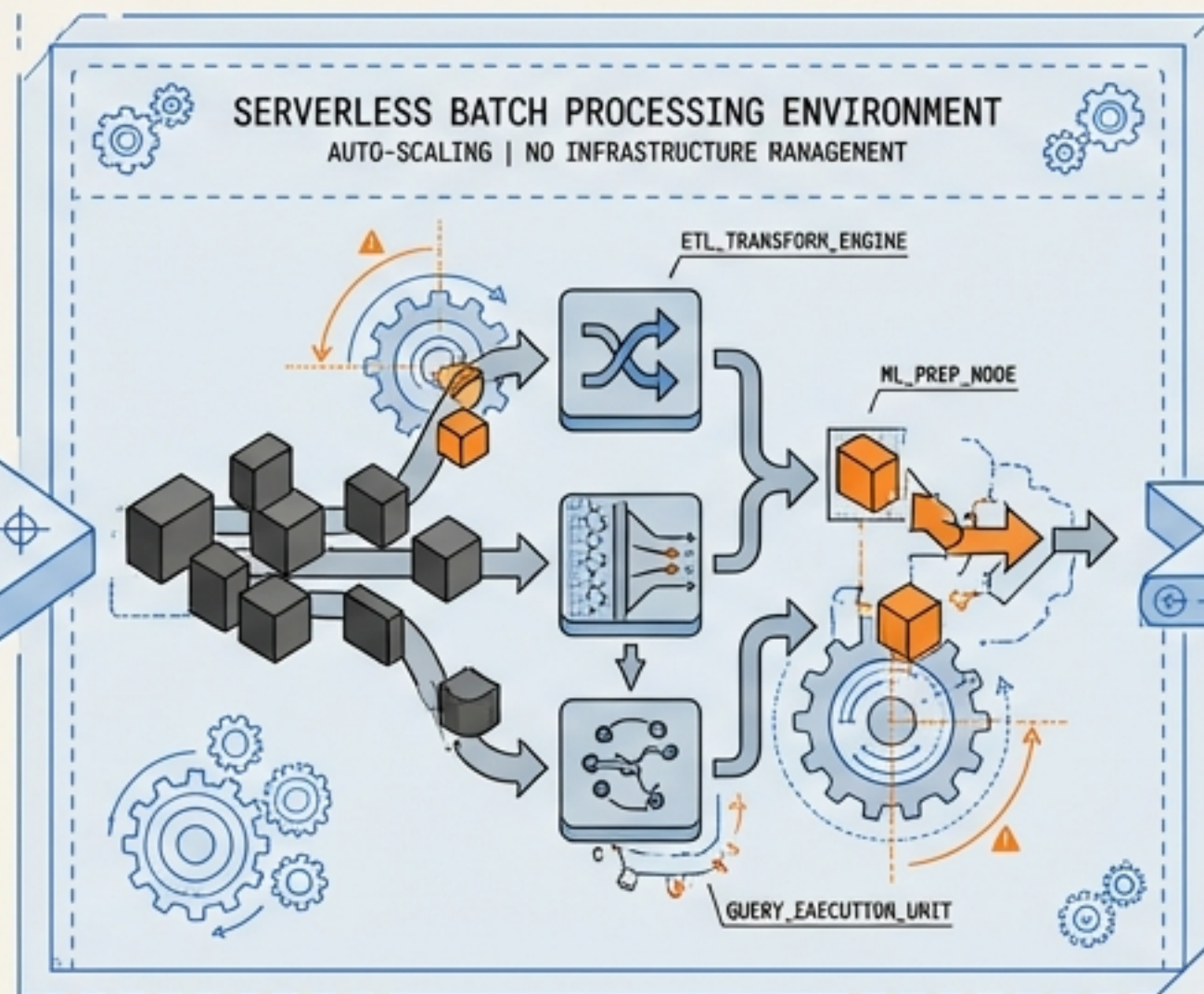


SERVERLESS
for Apache SPARK

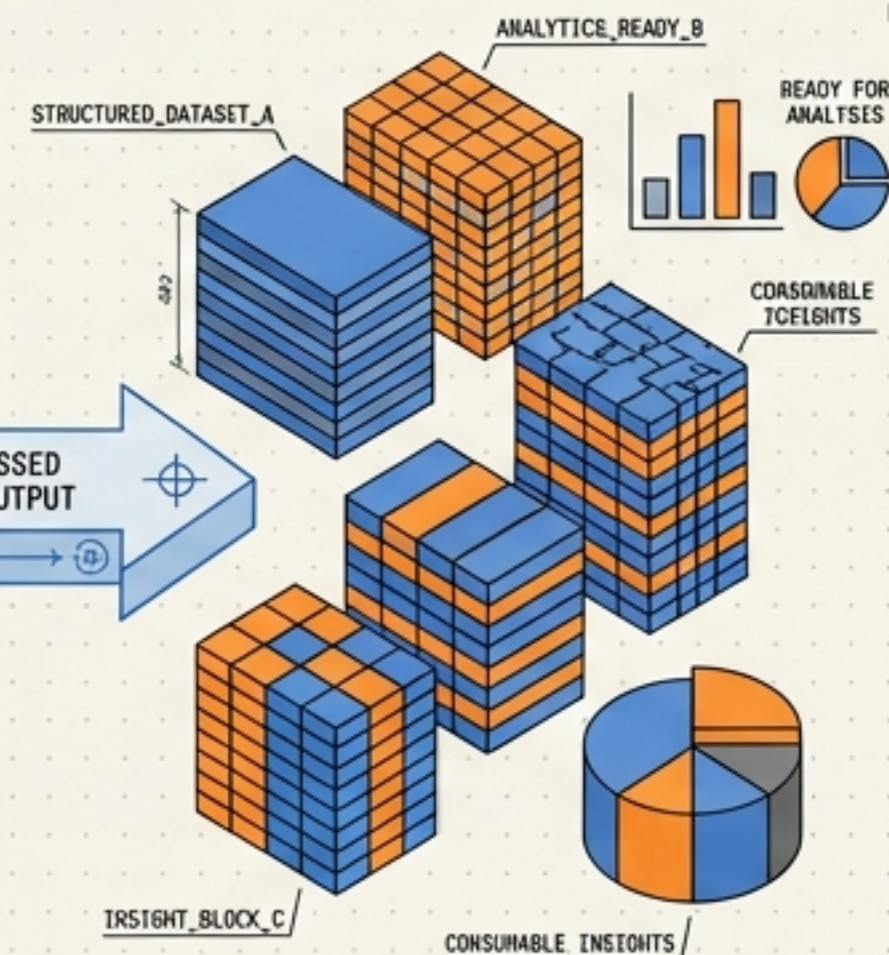
ZONE 01: RAW DATA INGESTION



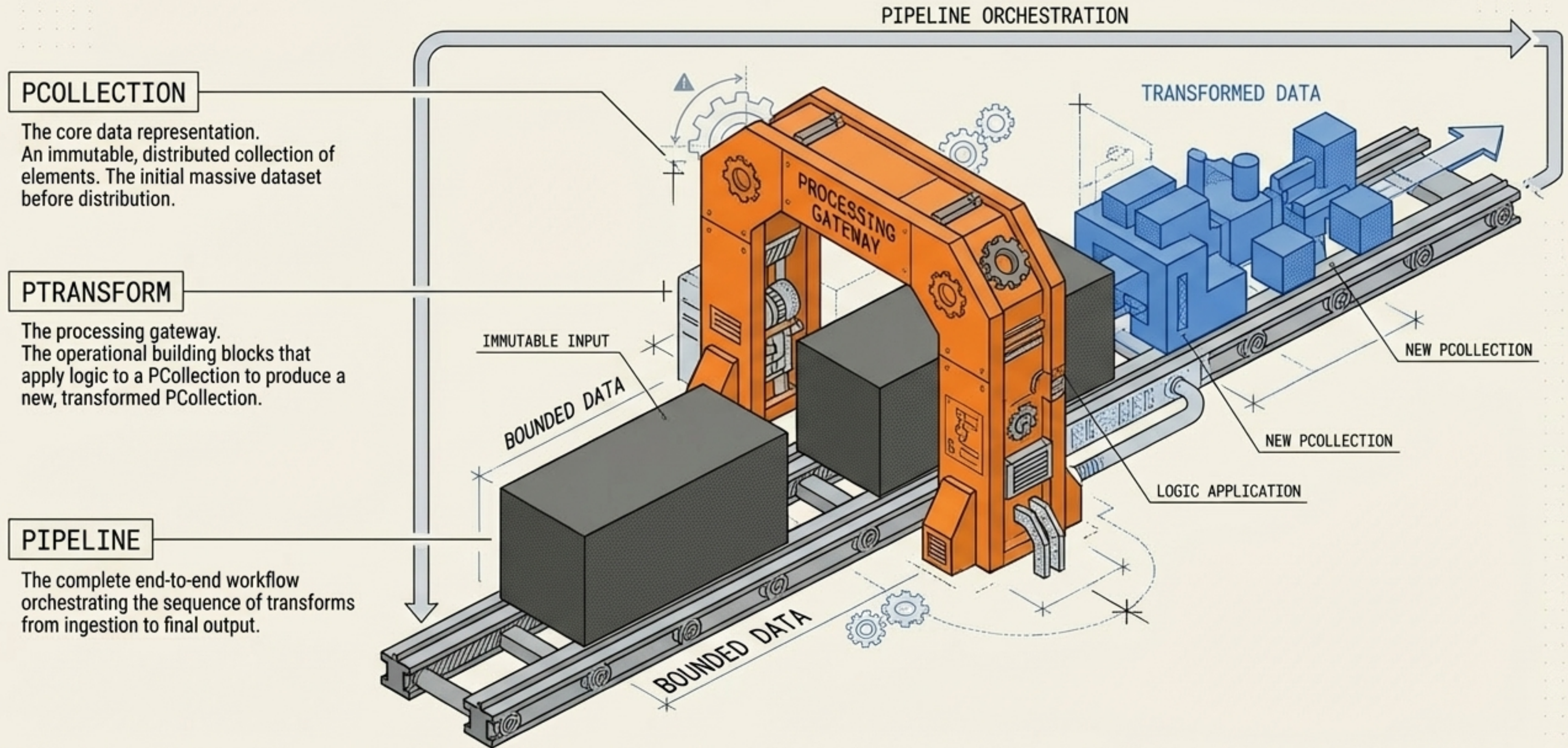
ZONE 02: SERVERLESS REFINERY & PROCESSING



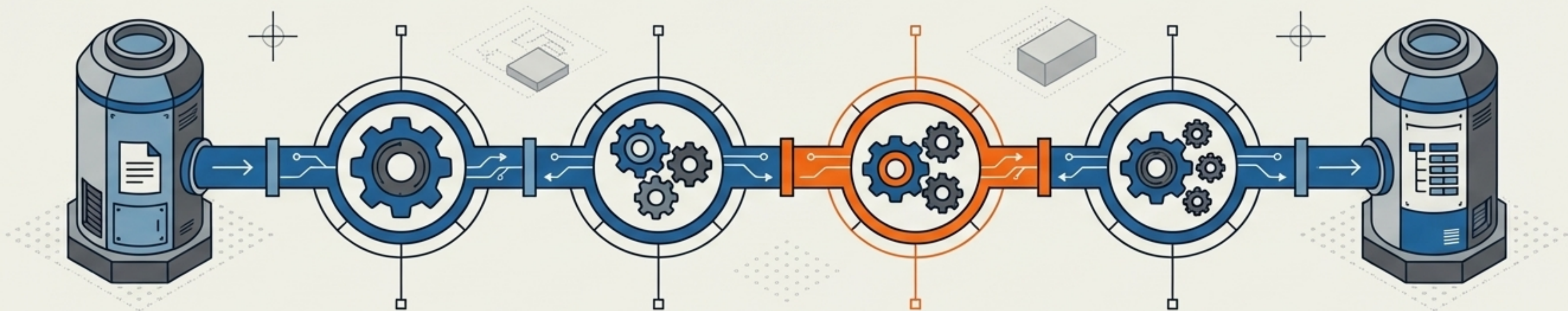
ZONE 03: STRUCTURED INSIGHTS & ANALYTICS



ENGINE 1: DATAFLOW & THE APACHE BEAM PRIMITIVES



THE JOURNEY OF A DATA RECORD: WORD COUNT PIPELINE



INPUT:

CLOUD STORAGE
TEXT FILE
('KING LEAR')

NODE 1 (READ)

CREATE
PCOLLECTION
OF INDIVIDUAL
LINES

NODE 2 (SPLIT PARDO)

CUSTOM REGEX
SPLITS LINES
INTO WORDS IN
PARALLEL

NODE 3 (LOWERCASE)

STANDARDIZE
TEXT TO PREVENT
DUPLICATE
COUNTS

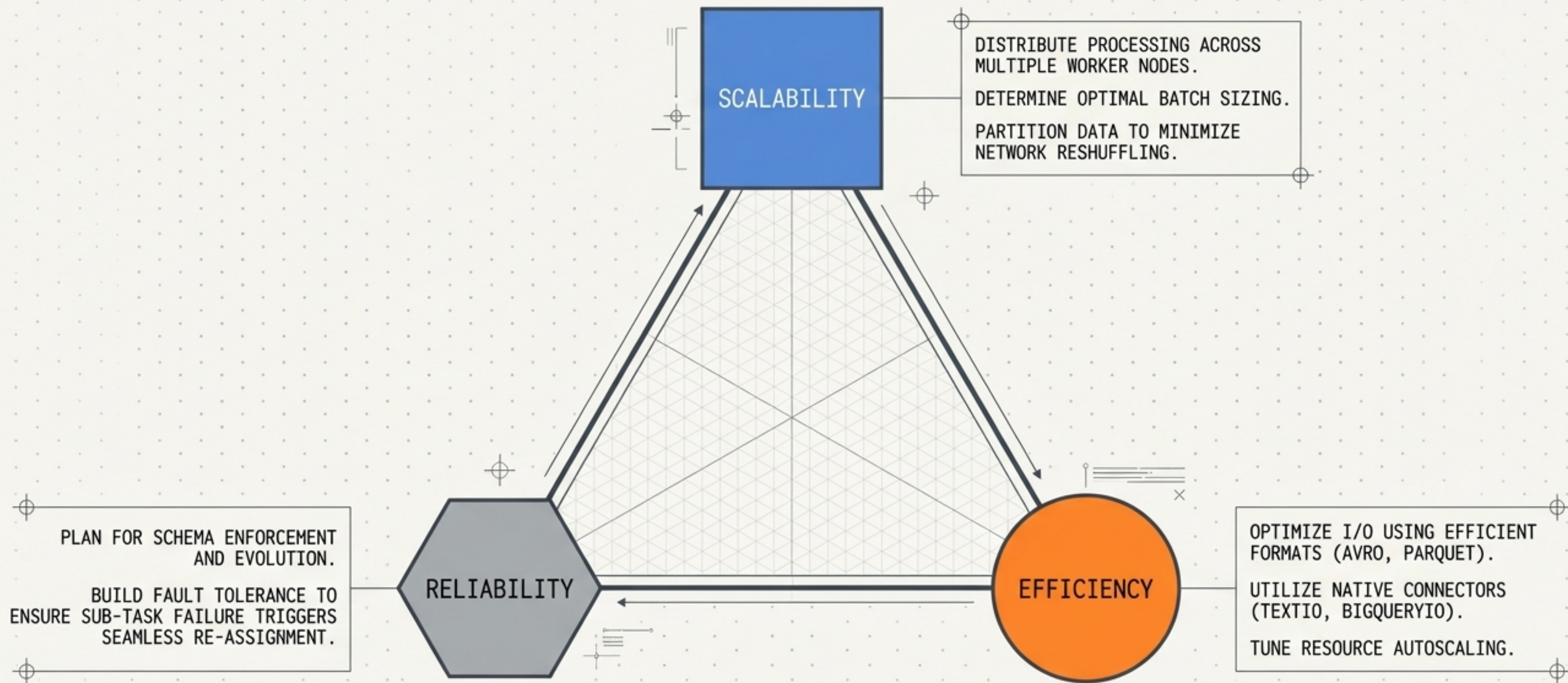
NODE 4 (GROUP & SUM)

AGGREGATE
COUNTS FOR
EACH UNIQUE
WORD

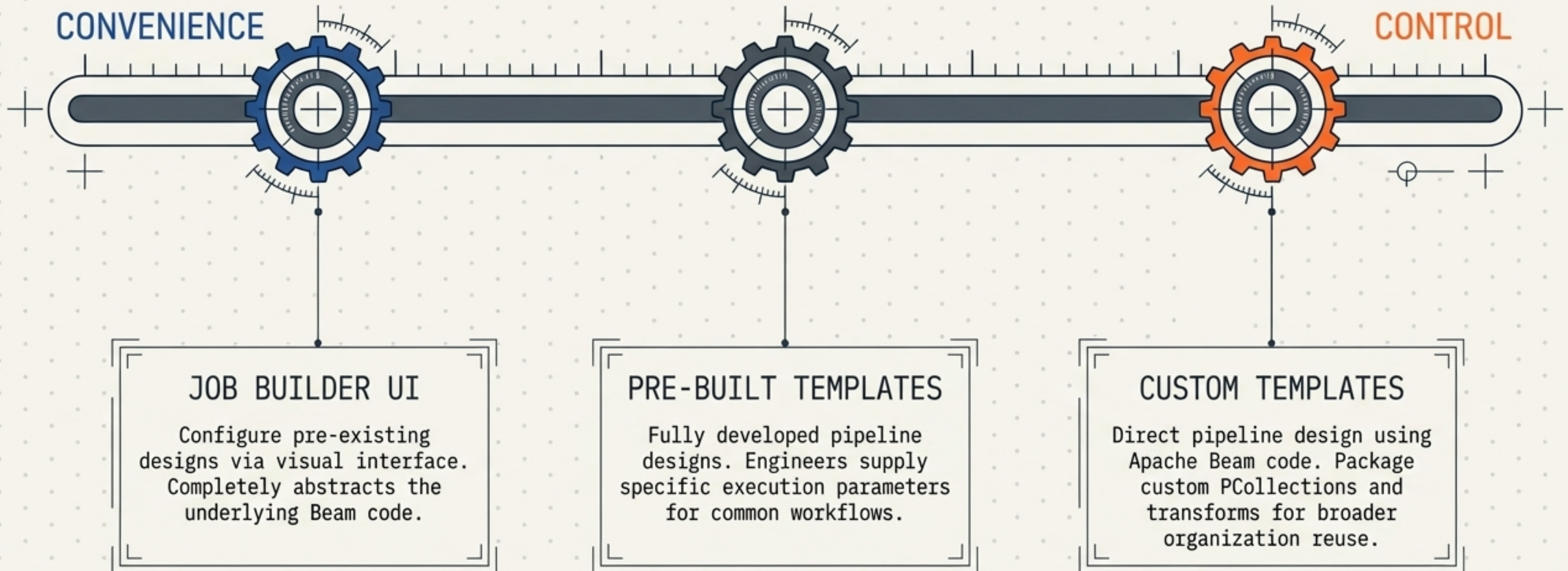
OUTPUT:

CLOUD STORAGE
BUCKET

DATAFLOW ARCHITECTURAL DESIGN PRINCIPLES

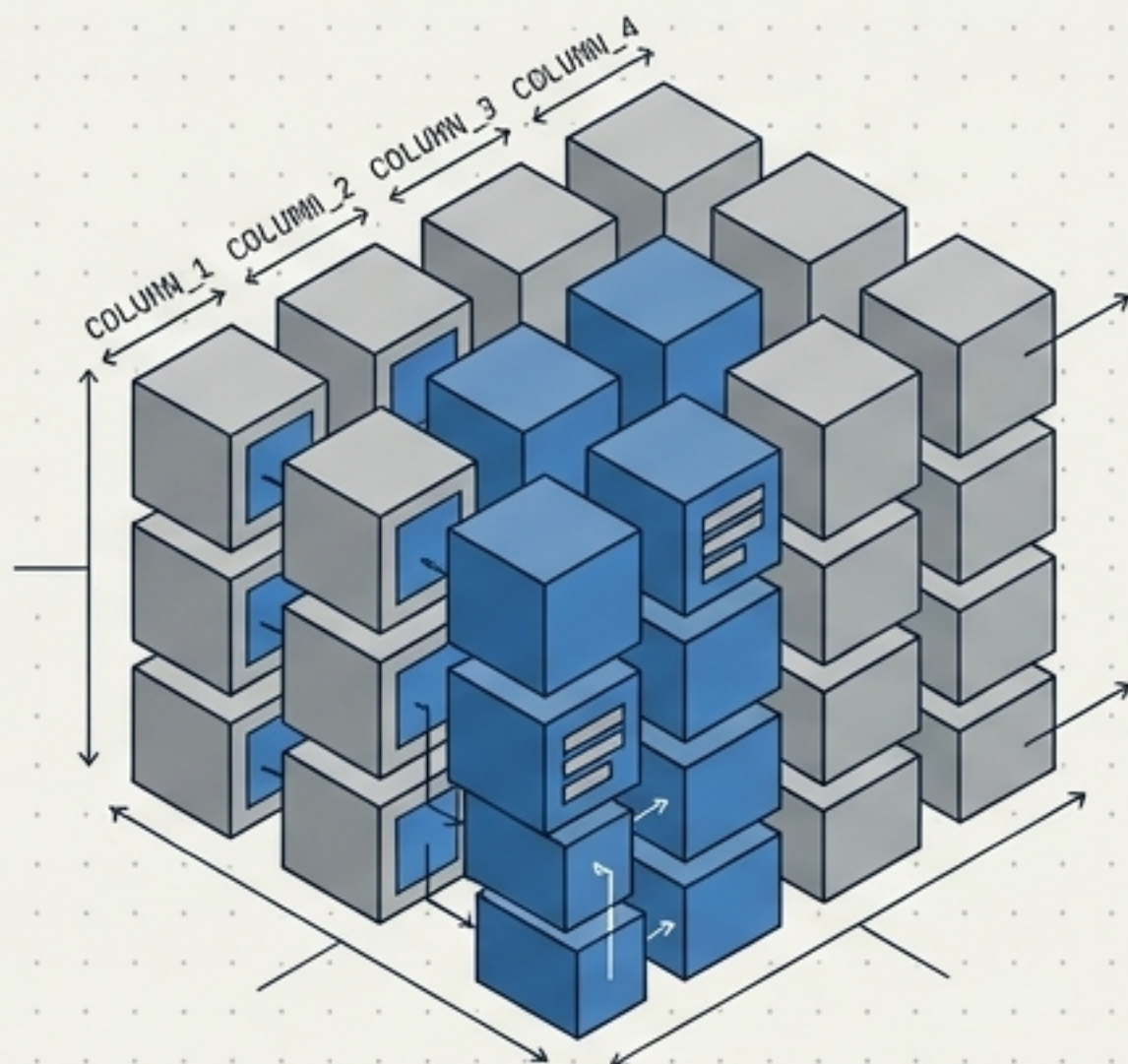


THE MODULARITY SPECTRUM: DATAFLOW TEMPLATES



ENGINE 2: SERVERLESS FOR APACHE SPARK ABSTRACTIONS

DATA STRUCTURES



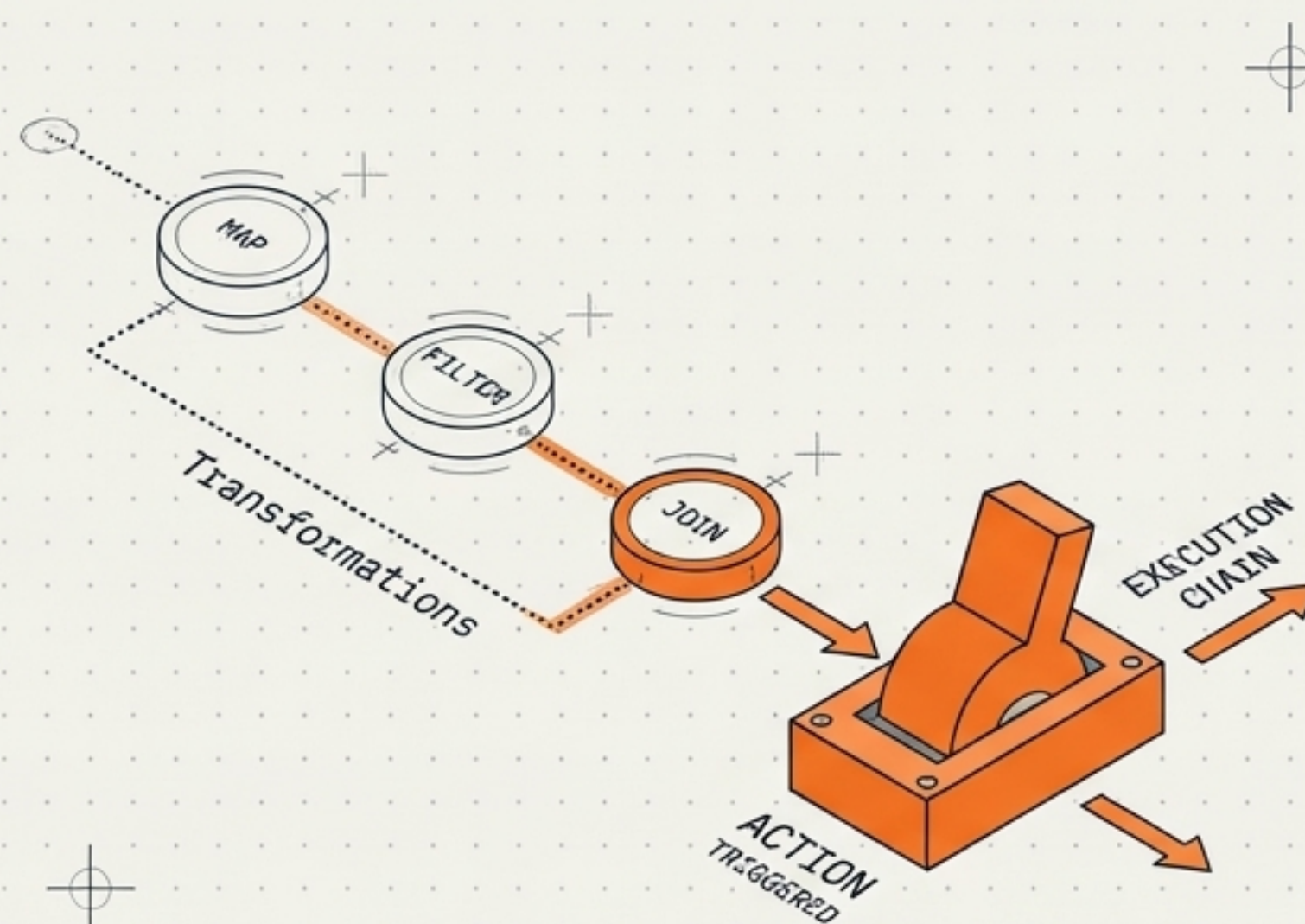
DATAFRAMES:

Structured data organized into named columns. Highly optimized with rich APIs. Integrates with Spark SQL.

DATASETS:

Advanced abstraction combining DataFrame structure with type-safety for error prevention.

THE EXECUTION TRIGGER



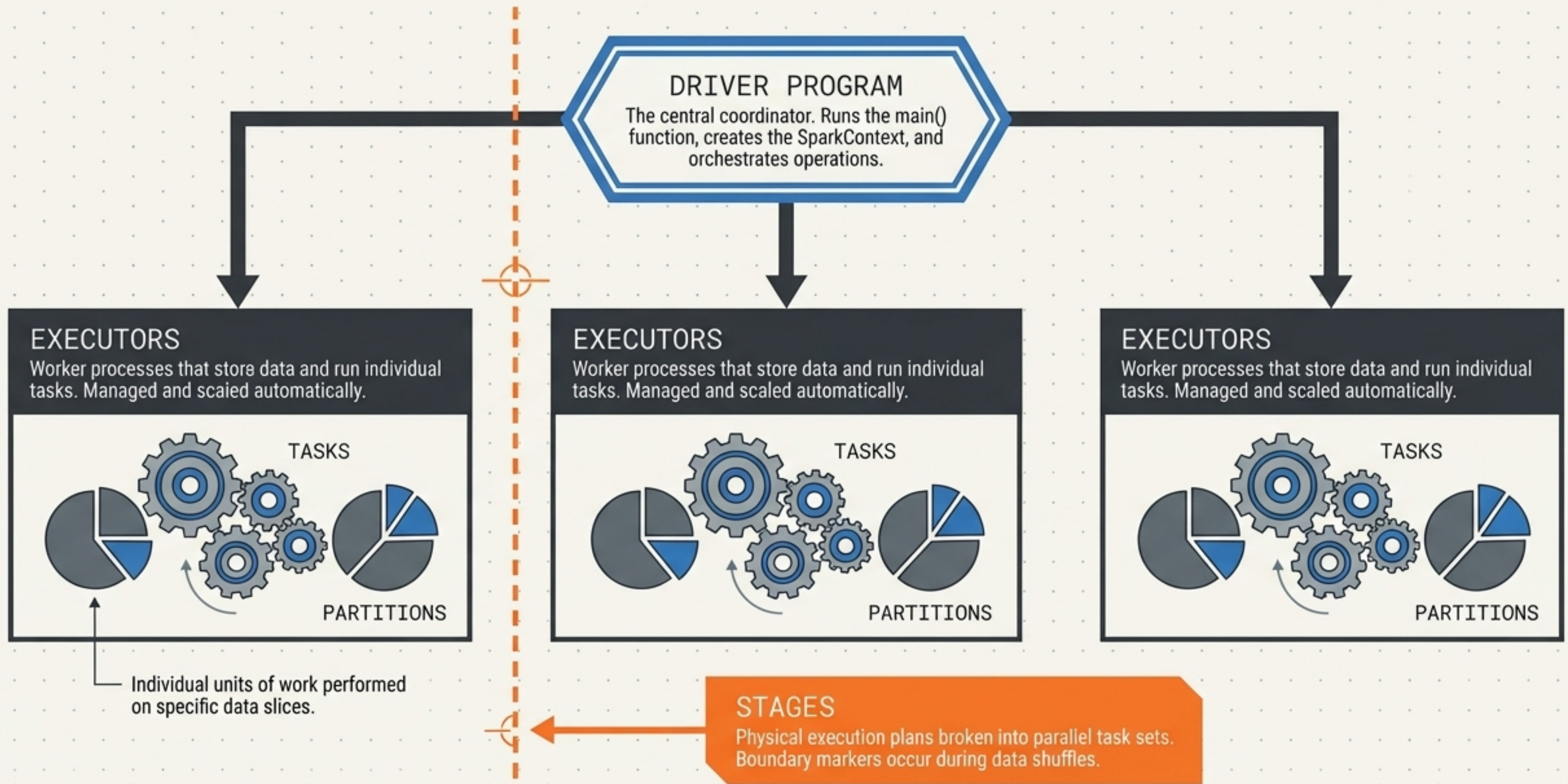
TRANSFORMATIONS:

Operations (map, filter, join) that create new abstractions. They are lazy-waiting in standby.

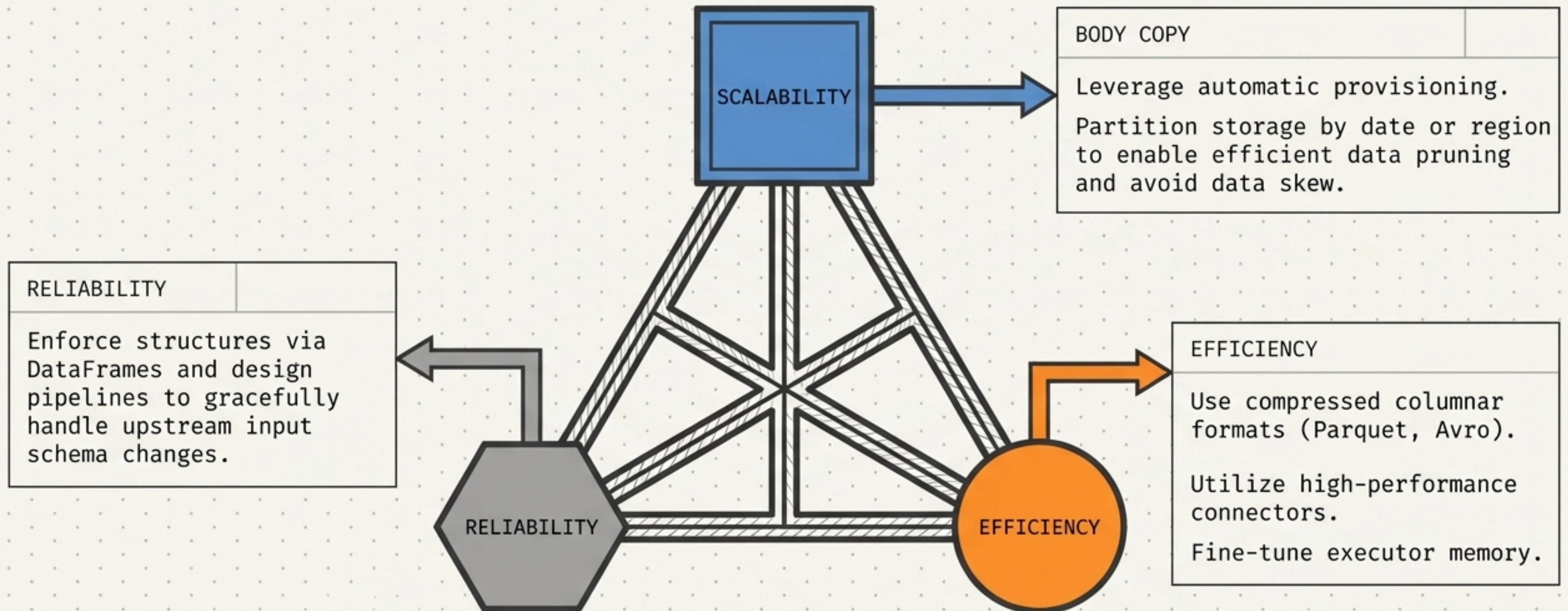
ACTIONS:

Operations (count, collect, write) that actually trigger the execution of the entire transformation chain.

SPARK EXECUTION MODEL NODE MAP



SPARK ARCHITECTURAL DESIGN PRINCIPLES



UNIVERSAL FAULT TOLERANCE: DEAD LETTER QUEUES

THE RISK

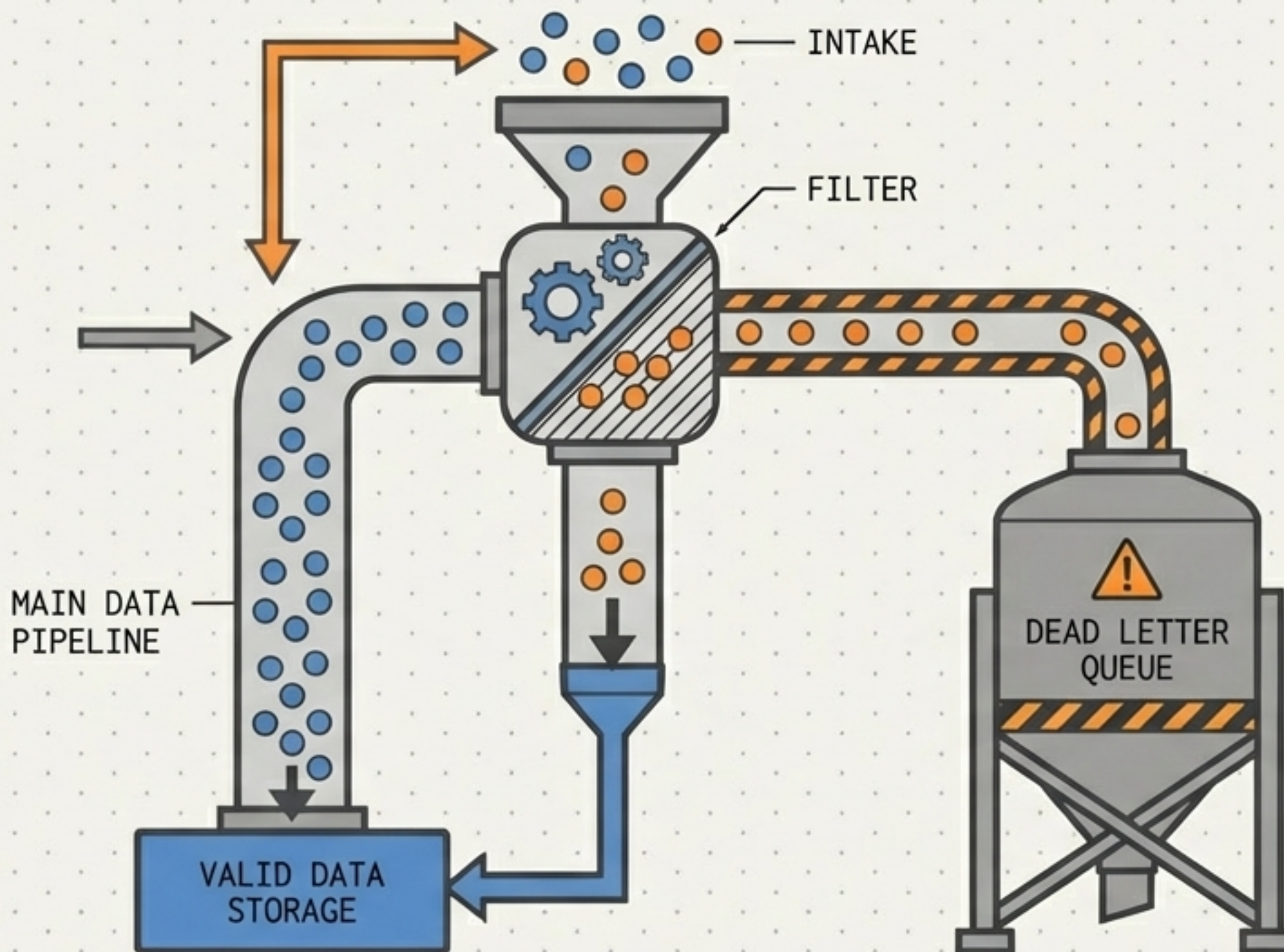
Corrupt or unparseable records can crash an entire batch job if unhandled.

THE DLQ ARCHITECTURE

Robust pipelines implement routing mechanisms to catch erroneous records and side-output them to a Dead Letter Queue.

THE RESULT

Main pipeline continues processing valid data without interruption, while bad data is quarantined for inspection.



SPARK MODULARITY: PARAMETERIZED APPLICATIONS

THE SERVERLESS ADVANTAGE

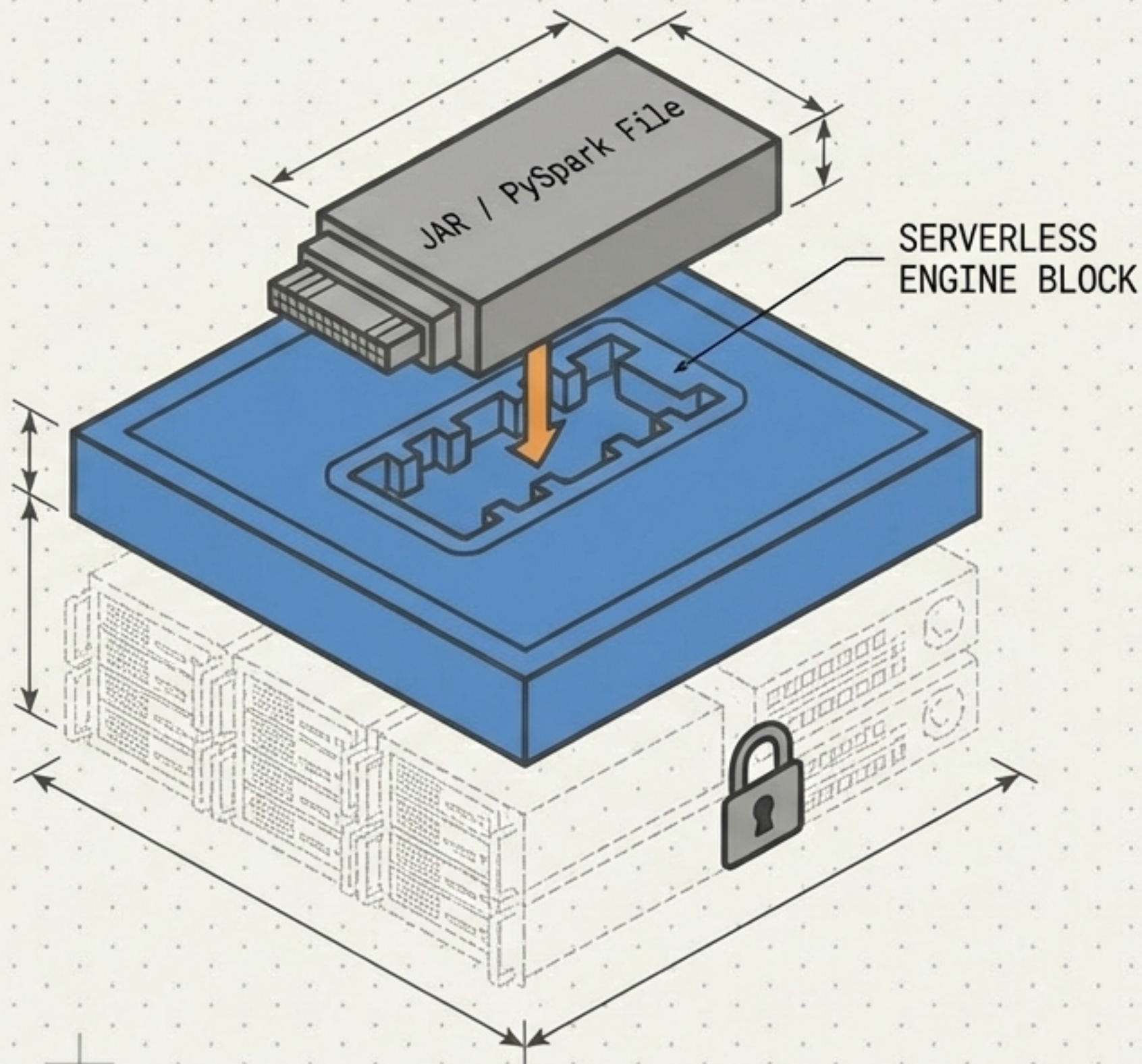
Developers focus purely on Spark application logic. The service eliminates cluster setup and auto-scales compute resources.

PARAMETERIZED TEMPLATES

Pre-packaged applications (Scala/Java JARs or Python files) with defined input parameters.

THE BUSINESS IMPACT

Standardizes workflows across teams, allows non-developers to launch complex jobs, and accelerates deployment.

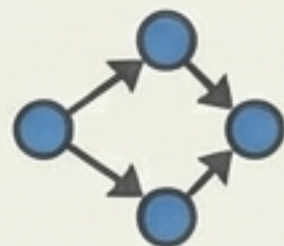


THE ENGINE SPEC SHEET

DATAFLOW

SPARK

CORE
ABSTRACTION



Unified Beam SDK;
transformation DAGs

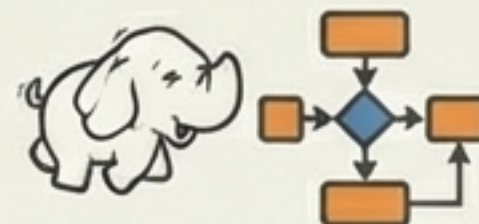


Native Apache Spark APIs;
DataFrames/Datasets

PRIMARY USE
CASE



Cloud-native pipelines,
complex ETL, real-time
streaming



Migrating Hadoop workloads, ML
workflows using Spark libraries

OPERATIONAL
MODEL

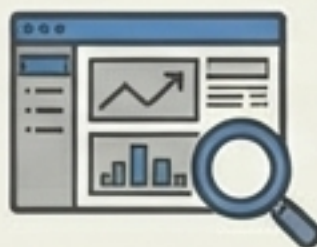


Fully serverless,
automated worker
management/rebalancing



On-demand provisioning,
management, and scaling of
clusters

ENGINEER
EXPERIENCE



Focus on pipeline logic;
Dataflow UI monitoring



Leverage existing Spark
expertise; Spark History
Server UI

THE ENGINE SELECTION MATRIX

